

# prompt\_manuel

---

title: “Case 6a — Faire un prompt (1)” subtitle: “Construire un prompt à partir de sorties statistiques” author: “SeRiouS” format: beamer: aspectratio: 169 theme: Madrid colortheme: seahorse fontsize: 10pt execute: echo: true eval: false ———

## Objectif de la case

Dans la case précédente, nous avons transformé des sorties statistiques en objets R :

```
texte_linearmodel  
texte_aovsum
```

Dans cette case, nous allons utiliser ces objets pour construire un premier prompt manuellement.

L’objectif est de passer de :

[ sortie statistique capturée      prompt organisé]

## Idée centrale

Un prompt utile pour l’interprétation statistique ne doit pas être une question vague.

Il doit contenir :

- un contexte ;
- les variables analysées ;
- la sortie statistique à interpréter ;
- des consignes explicites ;
- des limites d’interprétation.

Le modèle de langage reçoit ainsi un matériau contrôlé.

## Pourquoi construire un prompt dans R ?

Construire le prompt dans R permet de conserver une chaîne reproductible :

```
analyse statistique  
→ sortie capturée  
→ texte organisé  
→ prompt  
→ interprétation
```

Le prompt devient un objet R comme un autre.

Il peut donc être inspecté, modifié, sauvegardé ou transmis à une fonction.

## Deux fonctions importantes : `paste()` et `cat()`

La case introduit deux fonctions très simples mais essentielles :

```
paste()  
cat()
```

Elles n'ont pas le même rôle.

| Fonction             | Rôle                                        |
|----------------------|---------------------------------------------|
| <code>paste()</code> | construit une chaîne de caractères          |
| <code>cat()</code>   | affiche proprement un texte dans la console |

## Construire un texte avec `paste()`

Exemple :

```
exemple_texte <- paste(  
  "Première ligne",  
  "Deuxième ligne",  
  "Troisième ligne",  
  sep = "\n"  
)
```

Ici, `paste()` rassemble plusieurs morceaux de texte dans un seul objet.

L'argument :

```
sep = "\n"
```

indique que les éléments doivent être séparés par des retours à la ligne.

### Afficher un texte avec `cat()`

Une fois le texte construit, on peut l'afficher avec :

```
cat(exemple_texte)
```

Contrairement à `print()`, la fonction `cat()` affiche le texte de manière plus naturelle, en respectant les retours à la ligne.

C'est pratique pour visualiser un prompt dans la console.

### Premier prompt : régression linéaire

La case construit d'abord un prompt pour interpréter une sortie de régression linéaire :

```
prompt_linearmodel <- paste(
  "# Interprétation d une régression linéaire",
  "",
  "Contexte : questionnaire consommateur sur un produit alimentaire.",
  "Variable à expliquer : intention_achat.",
  "Variables explicatives : satisfaction et prix_percu.",
  "",
  "Sortie FactoMineR::LinearModel :",
  texte_linearmodel,
  "",
  "Consignes :",
  "- interpréter uniquement les résultats fournis ;",
  "- distinguer effet global et coefficients ;",
  "- ne pas confondre association et causalité ;",
  "- produire une interprétation pédagogique courte.",
  sep = "\n"
)
```

## Structure du prompt `LinearModel`

Le prompt contient plusieurs blocs :

| Bloc               | Rôle                                            |
|--------------------|-------------------------------------------------|
| Titre              | annoncer la tâche                               |
| Contexte           | situer les données                              |
| Variables          | préciser ce qui est expliqué et ce qui explique |
| Sortie statistique | fournir le matériau à interpréter               |
| Consignes          | encadrer la réponse attendue                    |

Cette structure réduit le risque d'une interprétation hors-sol.

### Pourquoi préciser les variables ?

Le modèle de langage ne doit pas deviner la situation.

On lui indique explicitement :

```
"Variable à expliquer : intention_achat."  
"Variables explicatives : satisfaction et prix_percu."
```

Cela permet de relier correctement les coefficients du modèle à la problématique.

### Pourquoi inclure la sortie capturée ?

Le prompt contient :

```
texte_linearmodel
```

Cet objet contient la sortie complète de `FactoMineR::LinearModel()` transformée en texte.

Le modèle ne reçoit donc pas seulement une consigne générale.

Il reçoit la sortie statistique produite dans R.

### Les consignes de prudence

Le prompt demande explicitement :

- interpréter uniquement les résultats fournis ;
- distinguer effet global et coefficients ;
- ne pas confondre association et causalité ;
- produire une interprétation pédagogique courte.

Ces consignes sont importantes.

Elles empêchent le modèle de produire une interprétation trop générale ou causaliste.

## Deuxième prompt : ANOVA avec interaction

La case construit ensuite un second prompt :

```
prompt_aovsum <- paste(
  "# Interprétation d une ANOVA avec interaction",
  "",
  "Variable à expliquer : satisfaction.",
  "Facteurs : type_produit et budget_constraint.",
  "",
  "Sortie FactoMineR::AovSum :",
  texte_aovsum,
  "",
  "Consignes :",
  "- identifier les effets significatifs ;",
  "- commenter prudemment l interaction ;",
  "- ne pas inventer de comparaisons post-hoc.",
  sep = "\n"
)
```

Ce prompt porte sur une analyse de variance.

## Structure du prompt AovSum

Le prompt précise :

| Élément            | Contenu                                     |
|--------------------|---------------------------------------------|
| Variable expliquée | satisfaction                                |
| Facteurs           | type_produit, budget_constraint             |
| Sortie statistique | texte_aovsum                                |
| Consignes          | effets significatifs, interaction, prudence |

Le modèle est guidé vers une lecture statistique contrôlée.

### **Pourquoi parler d'interaction avec prudence ?**

Dans une ANOVA, une interaction indique que l'effet d'un facteur peut dépendre du niveau d'un autre facteur.

Mais une interaction significative ne suffit pas à décrire toutes les comparaisons possibles.

C'est pourquoi le prompt précise :

```
ne pas inventer de comparaisons post-hoc
```

Le modèle doit rester au niveau des résultats fournis.

### **Afficher seulement un aperçu**

La case affiche seulement le début du prompt :

```
cat(substr(prompt_linearmodel, 1, 1500))
```

ou :

```
cat(substr(prompt_aovsum, 1, 1500))
```

Cela évite d'inonder la console avec un texte trop long.

Le prompt complet existe bien dans l'objet R.

### **Pourquoi tronquer l'affichage ?**

Le prompt peut être long parce qu'il contient une sortie statistique complète.

La console sert ici à vérifier que le prompt est correctement construit.

Il n'est pas nécessaire d'afficher tout le contenu pour comprendre la logique.

### **Objets créés dans la case**

À la fin de la case, deux objets importants sont disponibles :

```
prompt_linearmodel  
prompt_aovsum
```

Ces objets sont des chaînes de caractères.

Ils peuvent être transmis à un modèle de langage ou repris par une fonction d'automatisation.

### **Ce que contient `prompt_linearmodel`**

```
prompt_linearmodel
```

contient :

- le contexte du questionnaire ;
- la variable `intention_achat` ;
- les variables explicatives `satisfaction` et `prix_perçu` ;
- la sortie complète de `LinearModel()` ;
- des consignes d'interprétation.

C'est un prompt construit à partir d'une analyse statistique réelle.

### **Ce que contient `prompt_aovsum`**

```
prompt_aovsum
```

contient :

- la variable `satisfaction` ;
- les facteurs `type_produit` et `budget_constraint` ;
- la sortie complète de `AovSum()` ;
- des consignes sur les effets et l'interaction.

C'est un prompt adapté à une ANOVA.

## Ce que la case apprend

Cette case montre que le prompt peut être construit comme un objet structuré.

On ne tape pas simplement :

```
Interprète mon analyse.
```

On construit un texte qui organise :

```
contexte  
+ variables  
+ résultats statistiques  
+ consignes  
+ limites
```

## Lien avec la suite du plateau

À partir de cette case, deux chemins deviennent possibles :

1. automatiser cette construction avec EnTraineR ;
2. améliorer le prompt pour éviter de coder en dur les noms de variables.

Dans les deux cas, le principe reste le même :

[ sortie statistique      prompt contrôlé ]

## Message clé

Un prompt robuste ne vient pas seulement d'une bonne formulation.

Il vient surtout d'un matériau bien préparé.

```
sorties R  
→ texte capturé  
→ prompt structuré  
→ interprétation contrôlée
```

La qualité du prompt dépend donc de la qualité des objets construits en amont.

## À retenir

Dans cette case :

```
paste()
```

sert à construire le prompt.

```
cat()
```

sert à l'afficher.

```
texte_linearmodel
```

et

```
texte_aovsum
```

sont les matériaux statistiques injectés dans le prompt.

```
prompt_linearmodel
```

et

```
prompt_aovsum
```

sont les objets disponibles pour la suite.